

by-step guide.

Step 1. Install PySpark: Run the following code in a Colab cell to install PySpark:

```
! pip install pyspark
```

Step 2. Environment setup: Set the Spark home environment variables.

```
import os
import sys
# os.environ["SPARK_HOME"] = "/content/
spark-3.2.1-bin-hadoop3.2"
```

Step 3. Extract Spark: Download and extract the Spark binary. You can do this using the following code:

```
!wget -q https://d1cdn.apache.org/
spark/spark-3.2.1/spark-3.2.1-bin-
hadoop3.2.tgz
!tar xf spark-3.2.1-bin-hadoop3.2.tgz
!pip install -q findspark
!pip install pyspark
!pip install py4j
```

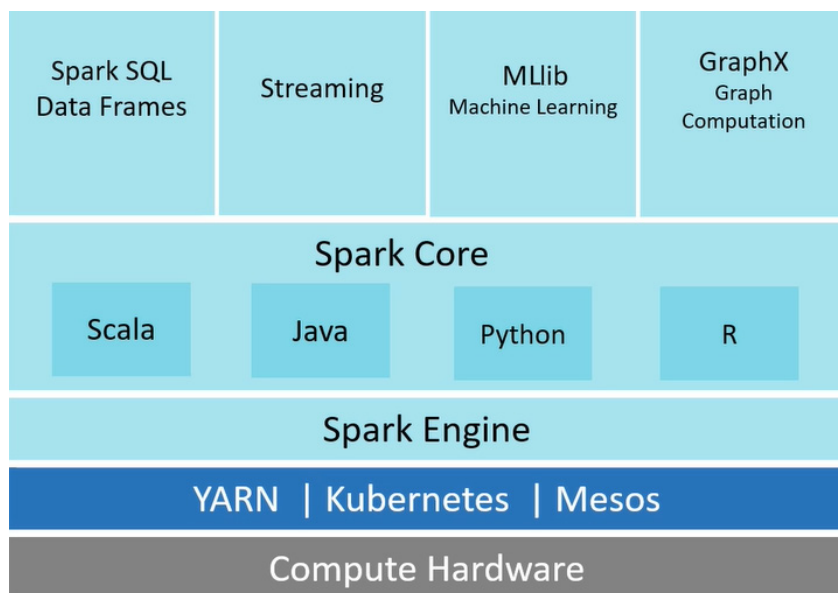


Figure 2: Key components of the Spark ecosystem

Step 4. Start Spark session: Start a Spark session in your Colab notebook. This step initialises Spark and allows you to use PySpark:

```
import pyspark
```

```
from pyspark.sql import DataFrame,
SparkSession
```

```
from typing import List
import pyspark.sql.types as T
import pyspark.sql.functions as F
```

Table 1: Comparison of Apache Spark with other popular tools

Feature	Apache Spark	Apache Hadoop	SQL	Pandas
Primary use	Big Data processing, analytics, machine learning	Distributed storage and processing	Querying and managing relational databases	Data manipulation and analysis in Python
Programming model	Scala, Java, Python, R	Java	Relational database management system (RDBMS)	Python
Fault tolerance	Provides fault tolerance through lineage and RDDs	Provides fault tolerance through replication and checkpointing	Depends on the DBMS and its configuration	Limited fault tolerance mechanisms
Performance	In-memory processing, faster than Hadoop	Slower than Spark, disk-based processing	N/A	Fast, but limited by memory and single machine
Ease of use	High	Lower than Spark, complex MapReduce programming	High for querying, moderate for complex tasks	High
Machine learning	Built-in MLlib library	Can integrate with various machine learning tools	Limited to querying, no built-in ML capabilities	Can integrate with machine learning libraries